Effectiveness of a One-Man Offense in the NBA

Vanhi Chava

2026-03-23

Problem Formulation

The problem in this experiment is the idea of a "one-man" offense, where in the NBA, one star player controls the majority of the team's possessions. This has become more of a prominent strategy in modern NBA, where high individual output has majorly influenced a team's wins and losses. While the high individual output (such as points and assists) is mainly positive, it typically comes at the cost of predictable team play and potentially high turnovers when only trying to keep the ball in one player's hands.

As a prediction task, I framed this as a binary classification problem. If we are given the individual performance data of the player who we know to be the star of the team (where we classify this based on number of minutes played and usage), can we determine whether a team wins or loses? I will focus on a non-linear model that I will train based on the star player's metrics to identify specific areas where increases in variables such as Turnovers or Usage will negatively impact a team's probability of a win.

The main people who benefit from this explanation are the NBA team coaching staffs and the those who are part of the front office. For the coaches, having these post-hoc explanations are vital to selecting the right rotational choices and when to rotate players out who have played too much. This can allows coaches to maximize the team's efficiency by noticing when players tend to become fatigued. For the front office, these explanations are what will guide which players are selected for a team. For example, if the model reveals that a star player's high assists are less powerful in predicting team wins than a metric like low turnovers, then the front office may look to select more role players who make less offensive mistakes (less turnovers) then players who are more skilled on offense (high points and assists).

I accessed the data from Kaggle Datasets, specifically at this link: https://www.kaggle.com/datasets/eduardopalmieri/nba-player-stats-season-2425. I used the 2024-2025 data specifically to see how this issue of over-dependence on star players may have started around this time, based on my own NBA knowledge.

As for preliminary data processing, I focused first on restricting the dataset to players who have played more than 30 minutes in the games to focus on the star players. Then, I converted the "Result" column (having values W/L) into a binary column.

I also needed a way to measure a player's usage rate in a game, which I measured using the formula mentioned in the article https://sportsanalytics.studentorg.berkeley.edu/articles/conceptions-usage.html where the formula is cited through sports analysts in ESPN. I used the contemporary method of calculating the usage rate which is the formula: $100*(FGA_{star}+0.44*FGA_{star}+TOV_{star})*(Minutes_{team}/5)/((FGA_{team}+0.44*FTA_{team}*TOV_{team})*(Minutes_{star}))$. Finally, I only selected important variables, removing unecessary variables so the model will not be able to memorize which teams are currently bad and focus on the relationship between the player and the outcome.

Dataset Preparation:

```
nba_df <- read.csv("database_24_25.csv")

nba_processed_df <- nba_df %>%
```

```r
  # only players with "Minutes Played" > 30
  filter(MP > 30) %>%

  # convert the "Res" column of W/L into binary
  mutate(Win = ifelse(str_detect(Res, "W"), 1, 0)) %>%

  # adding the "Usage_Rate" as a variable (contemporary method)
  group_by(Tm, Data) %>%
  mutate(tm_FGA = sum(FGA, na.rm = TRUE),
         tm_FTA = sum(FTA, na.rm = TRUE),
         tm_TOV = sum(TOV, na.rm = TRUE),
         tm_MIN = sum(MP, na.rm = TRUE)) %>%
  ungroup() %>%
  mutate(Usage_Rate = (100 * (FGA + 0.44 * FTA + TOV) * (tm_MIN / 5)) / ((tm_FGA + 0.44 * tm_FTA + tm_T

  # only select needed metrics
  dplyr::select(Win, Usage_Rate, PTS, AST, TOV, STL, BLK, TRB, FG., X3P., FT., PF, GmSc, MP) %>%
  drop_na()
```

Model Training

I am training an XGBoost model which is a tree ensemble method. Instead of using a single decision tree, the XGBoost model uses the process of boosting, where it will build a series of fairly weak trees where each subsequent tree is trained to predict the residuals of the previous tree.

The main hyperparameters are *eta*, *max_depth*, and *nrounds*. Firstly, *eta* represents the learning rate and is set to 0.1 in order to scale each tree's contribution and prevent overfitting. The second hyperparameter is *max_depth* which is set to 6 in order to allow the model to capture the non-linear, complex relationships between the usage and the win status. Finally, *nrounds* is set to 500 iterations as the number of boosting iterations. It is also important to note that I set the *objective = "binary:logistic"* in order to get an output of probabilities that would directly correspond to the win/loss classification.

The hyperparameter selection was done through manual tuning and validation. I initially set a medium learning rate (eta = 0.1) to ensure that the boosting process was more on the conservative side, where I would need more iterations as a trade-off to stabilize the predictions across the dataset. The max_depth being around 6 allows me to control the complexity of up to 6-way interactions, and for validation, I used a 80/20 split where I specifically monitored the training error and out-of-sample error to make sure that the model generalizes well to unseen data.

```r
set.seed(122)

# manually splitting data 80/20 (train/test)
n <- floor(0.8 * nrow(nba_processed_df))
train_idx <- sample(seq_len(nrow(nba_processed_df)), size = n)

# training and test data frames
train_df <- nba_processed_df[train_idx, ]
test_df <- nba_processed_df[-train_idx, ]

# preparing the DMatrix
dmatrix_train <- xgb.DMatrix(data = as.matrix(train_df %>% dplyr::select(-Win)), label = train_df$Win)
dmatrix_test <- xgb.DMatrix(data = as.matrix(test_df %>% dplyr::select(-Win)), label = test_df$Win)
```

```r
# train the model
final_xgb_model <- xgb.train(params = list(max_depth = 6,
                                           eta = 0.1,
                                           objective = "binary:logistic"),
                             data = dmatrix_train,
                             nrounds = 500,
                             verbose = 0)

# calculation for out-of-sample accuracy
predictions_probability <- predict(final_xgb_model, dmatrix_test)
binary_predictions <- ifelse(predictions_probability > 0.5, 1, 0)
oos_accuracy <- mean(binary_predictions == test_df$Win)
oos_accuracy
```

```
## [1] 0.5629478
```

The out-of-sample prediction accuracy is approximately 0.5629. This is only slightly above 0.50 which would be random guessing, telling us that while the model captures some signal, predicting the outcomes of games by only using player statistics is very difficult.

Explanation 1

I will create CP profiles below. These profiles are a local explanation method, specifically for visualizing how the model's prediction changes when a single feature is changed. The scale of the explanation output is on the probability scale. While the XGBoost function operates using log-odds scale, the DALEX explainer specifically transforms these values between 0 and 1 for the probability scale (representing the probability of a game win).

```r
# CP explainer
cp_explainer <- explain_xgboost(
  model = final_xgb_model,
  data = train_df %>% dplyr::select(-Win),
  y = train_df$Win,
  label = "XGBoost",
  type = "classification"
)
```

```
## Preparation of a new explainer is initiated
##   -> model label       :  XGBoost
##   -> data              :  3907  rows  13  cols
##   -> data              :  tibble converted into a data.frame
##   -> target variable   :  3907  values
##   -> predict function  :  yhat.xgb.Booster  will be used (  default  )
##   -> predicted values  :  No value for predict function target column. (  default  )
##   -> model_info        :  package xgboost , ver. 3.2.0.1 , task classification (  default  )
##   -> model_info        :  type set to  classification
##   -> predicted values  :  numerical, min =  0.002854784 , mean =  0.5017282 , max =  0.9977138
##   -> residual function :  difference between y and yhat (  default  )
##   -> residuals         :  numerical, min =  -0.5865849 , mean =  -6.448314e-05 , max =  0.5165073
##   A new explainer has been created!
```

```r
# Visualizations 1: Usage Rate Furthest Point
# Visualization 2: Impact of Turnovers
pdp <- model_profile(cp_explainer, variables = c("Usage_Rate", "TOV"))
plot(pdp, geom = "profiles")
```

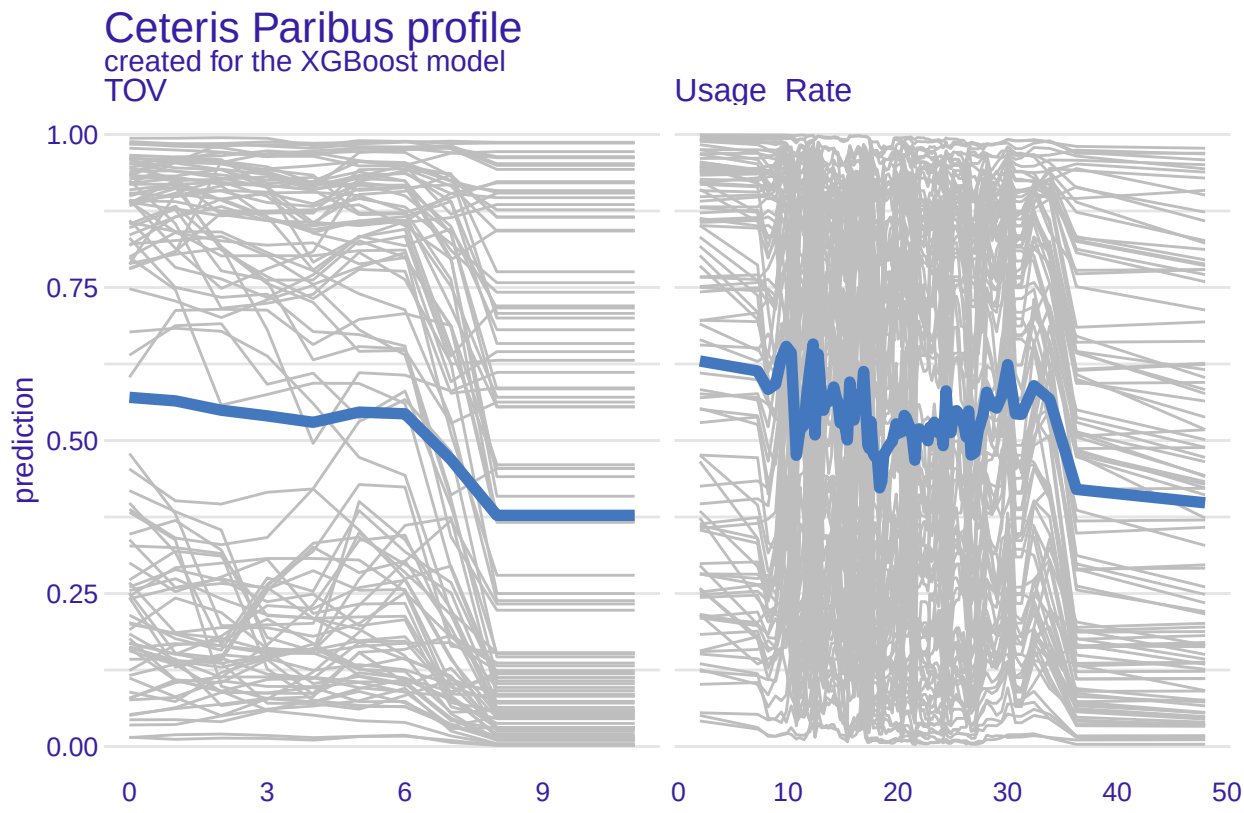

Figure 1: CP Profles for Turnovers and Usage Rate

For the first plot relating to turnovers, the CP profile is relatively stable between about 0 and 6 turnovers with a slight "v" fluctuation at 3.5 turnovers, but otherwise the win probability remains over 0.50. However, at around 6 turnovers, the probability has a major decline from about 0.50 to 0.30 and remains constant from there. This idea reveals a non-linear relationship with the idea that once you reach a certain number of turnovers, there is not a substantial chance to try to win the game. The model suggests that the star players have a slight buffer for mistakes where teams can remain competitive for the win between about 4-6 turnovers, but after that the threshold of 7 turnovers seems to have a major decreasing effect on the chance of a win. This showcases how a star player's negative impact could potentially outweigh an incredible offensive performance (important information to consider for head coaches especially).

For the second plot for Usage Rate, the CP profile starts fairly high at 0.62 probability and has fairly predictable oscillations until about 40% usage where a major dip occurs (to around 0.30 probability) and after a final increase, a linear decline begins. To start, the oscillation between about 10% and 35% usage suggests that the model might be sensitive to the difference between a role player and a secondary star player, where a player might be using too many possessions for a role player but not enough to fully carry the offense. The main point to notice is the major dip at around 40% usage which showcases the clear ceiling for efficiency. In the 2024-2025 NBA seasons, the model suggests that once a player fully controls around 40% of their team's plays, the team becomes statistically more likely to lose. As to why this happens,

reasons may include that the offense becomes predictable (because only one player is scoring) or the star player becomes exhausted.

A potential way that one could misread the output is that they could use the CP Profile as a strict guide, saying that if a player simply increases their usage in a game to the peak of the curve, then their team will win more. However, this is not true because in a game, there are various other factors that impact a team's win, such as Field Goal Percentage, Total Points, and more. I would correct this idea by explaining that the profiles are more "what-if" scenarios to see the logic of the model for a specific variable, not actual independent realities.

Explanation 2

In the second explanation, I created two plots using a SHAP method, one with a local explanation and one with a global explanation. For the local explanation, I used a waterfall plot which decomposes the prediction of a specific game to see the contributions of each individual feature. For the global explanation, I used a beeswarm plot which will show the SHAP values across the entire test dataset (all games) and show the overall significance of each feature. The output of this explanation is on the log-odds scale where SHAP values represent the contribution of each feature (additive) to the model before it is transformed into a probability. The XGBoost I used is a boosted version of logistic regression where the features are additive only in the log space.

```r
set.seed(124)

# specific features defined
features_needed <- c("Usage_Rate", "PTS", "AST", "TOV", "STL", "BLK", "TRB", "FG.", "X3P.", "FT.", "PF"

# shapviz object
shap_viz_obj <- shapviz(final_xgb_model, X_pred = as.matrix(test_df[, features_needed]))
first_game <- shap_viz_obj[1]

# Visualization 1: Local Waterfall Plot
sv_waterfall(first_game) +
  labs(title = "SHAP Waterfall Plot: J. Tatum (BOS vs. NYK)",
       x = "SHAP Value",
       y = "Feature")
```

```r
# Visualization 2: Global Feature Importance
sv_importance(shap_viz_obj, kind = "beeswarm") +
  labs(title = "SHAP Global Importance: NBA Star Players",
       x = "SHAP Value",
       y = "Feature")
```

For the local explanation plot, the waterfall plot specifically showcases key insights to the different features impact on team win probability through Jayson Tatum's individual performance. While he had a very high game score of 34 which pushed the prediction up by +2.13, his 36 points and 37.6 minutes played both show negative SHAP values (-0.697 and -0.594 respectively). This suggests that for this specific game where Tatum was a high-usage player, the model found that his scoring volume and time playing was actually less efficient than the average winning performance.

For the global explanation plot, the beeswarm plot shows how the features are ranked based on their average impact and each dot is its own game from the dataset. An important insight from the beeswarm plot is that the Usage Rate was the third most important global feature where the spread is even on the positive and

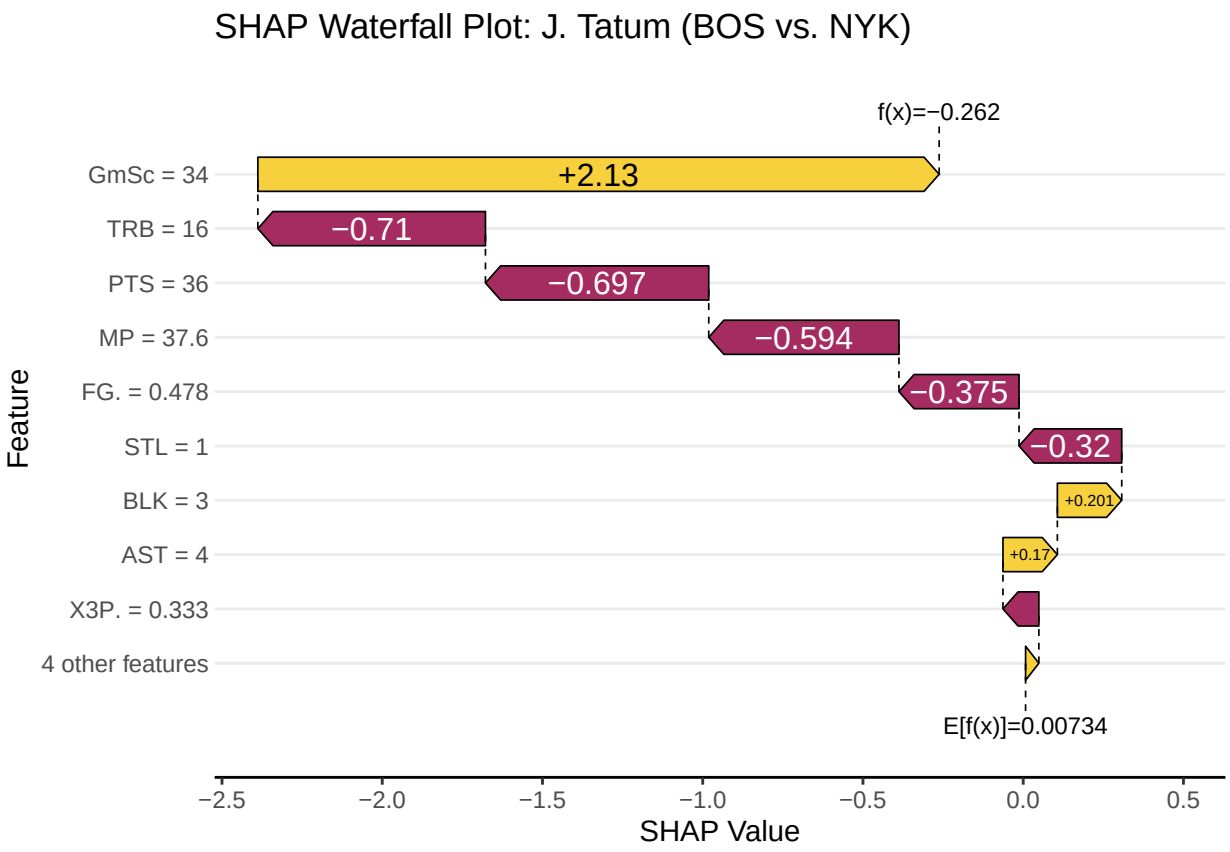

Figure 2: SHAP Waterfall Plot for Jayson Tatum Performance (BOS vs. NYK)

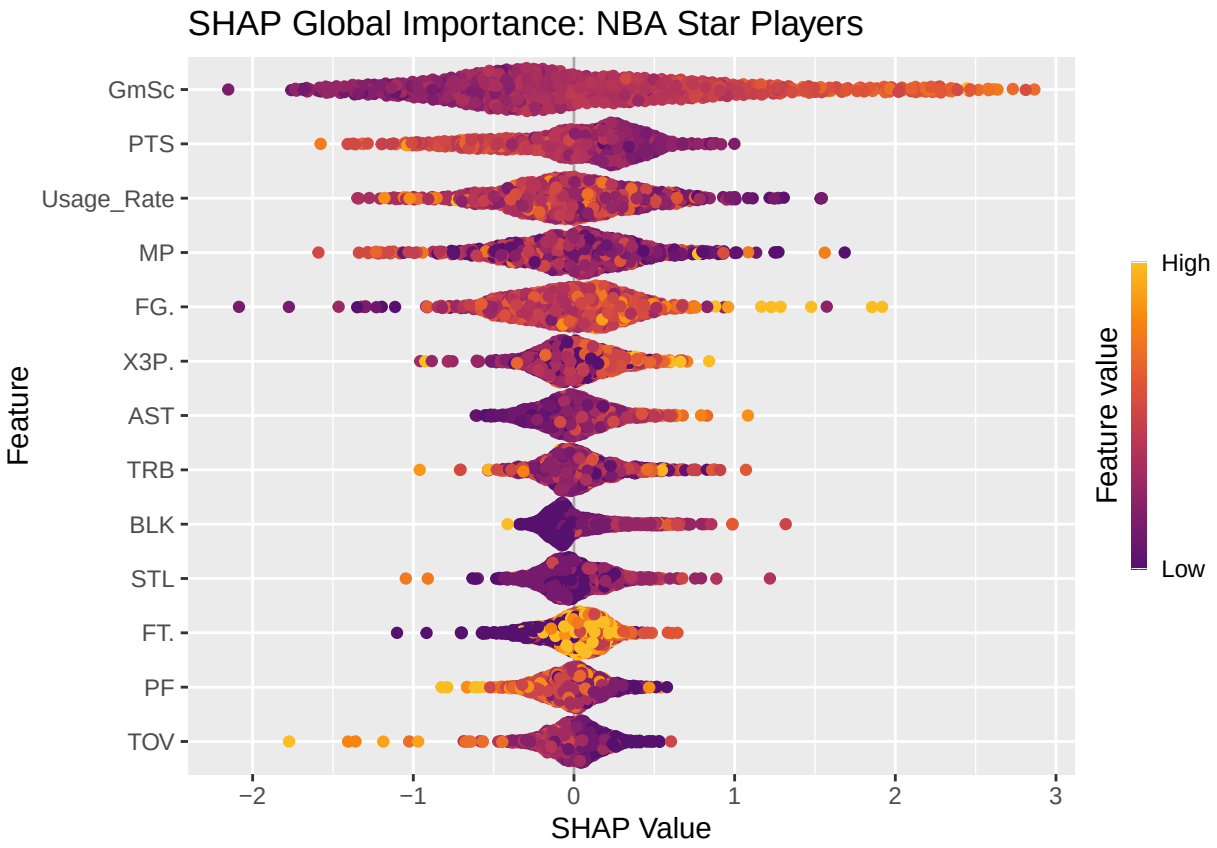

Figure 3: SHAP Global Feature Importance: Beeswarm Plot

negative sides of 0 with mid to high feature values. This confirms the idea that I had initially, that high usage of a star player does not automatically mean a win. High usage can have either a major positive or negative impact on a player's efficiency which is major risk for the team's efficiency.

One misinterpretation that one may have when reading the Waterfall plot is that they may look at certain features that had a negative SHAP value and immediately assume that these would hurt the team. For example, Tatum's 36 points had a negative SHAP value but this does not necessarily mean that scoring points is considered bad. In this case, the high number of points having a lower SHAP value needs to be taken in context with the other features. The 36 points essentially provided less information for the model's prediction of win probability than what was expected, meaning the model likely had accounted for scoring through the Game Score already. I would specifically make sure to explain that the SHAP values show marginal contribution of a feature given others and not to take the SHAP values shown as an absolute final takeaway without understanding context.

Method Description

Often times in modern sports analytics, complex models (like XGBoost) are very powerful at predicting wins but they are black box methods, meaning that we are unable to see how the model makes its decisions. This becomes difficult for the coaching staff and the front office to see why the model predicted the way it did. To bridge the gap between interpretablity and predictive power, I used SHAP methods, which is a method that will provide transparency for each game statistics for the players in the model and how they impact the win probability.

Imagine a specific basketball game where a team scored 120 points. If you want to know how much credit a specific player deserves for those points, you can't look at the individual box score for the full picture. You would need to look at how the team performed when the player was on the floor versus when they were off the floor to understand their true impact on team efficiency. SHAP applies this same idea to the model prediction. It calculates the significance of each of our features (each statistic measuring player performance) by testing the model's prediction among the combinations of other features. In this case, SHAP determined the contribution of each specific feature by measuring how the win probability changes when the feature is included versus excluded. This is helpful in ensuring that the credit is distributed fairly among all features, even when the features might be correlated (ex. scoring a lot of points usually means that a player's usage rate will be high).

The outputs of the method are SHAP values, which measures the impact of each of the features on the win probability (will be a positive impact or negative impact). The local explanation, which we see in the SHAP Waterfall Plot, starts with a single game and uses the league average win probability as a baseline. Going from there, it shows exactly how each statistic for a specific player added or subtracted value from the baseline until you get to the final prediction. In my plot, I found that Jayson Tatum's high number of points and high number of minutes (indicating high-usage) actually negatively impacted the team's win probability. On the other hand, the global explanation which is the SHAP Plot with a beeswarm type (looks initially more messy) averages the SHAP values across the entire 2024-2025 seasons and then the output identifies which features the model relies on the most. This will allow us to see, on average, if a high Usage Rate is a reliable predictor of wins or of risk. In this case, the Usage Rate was found to be important but did not automatically result in a win for a team, but rather a "high risk, high reward" type factor.

In the end, the SHAP methods can be an immense help in understanding how model decisions are made for front offices. Specifically, in my project, the SHAP methods revealed how a high usage rate of a star did not automatically result in a win, but rather would advise teams to be more cautious of fully relying on one player.

Checks

```r
# correlation matrix
correlation_matrix <- cor(train_df %>% dplyr::select(-Win))

# heatmap
cor_long_df <- as.data.frame(as.table(correlation_matrix))
ggplot(cor_long_df , aes(Var1, Var2, fill = Freq)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low = "red", high = "dodgerblue", mid = "white", midpoint = 0, limit = c(-1, 1),
  labs(title = "Predictor Correlation Heatmap",
       x = "Feature",
       y = "Feature") +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```

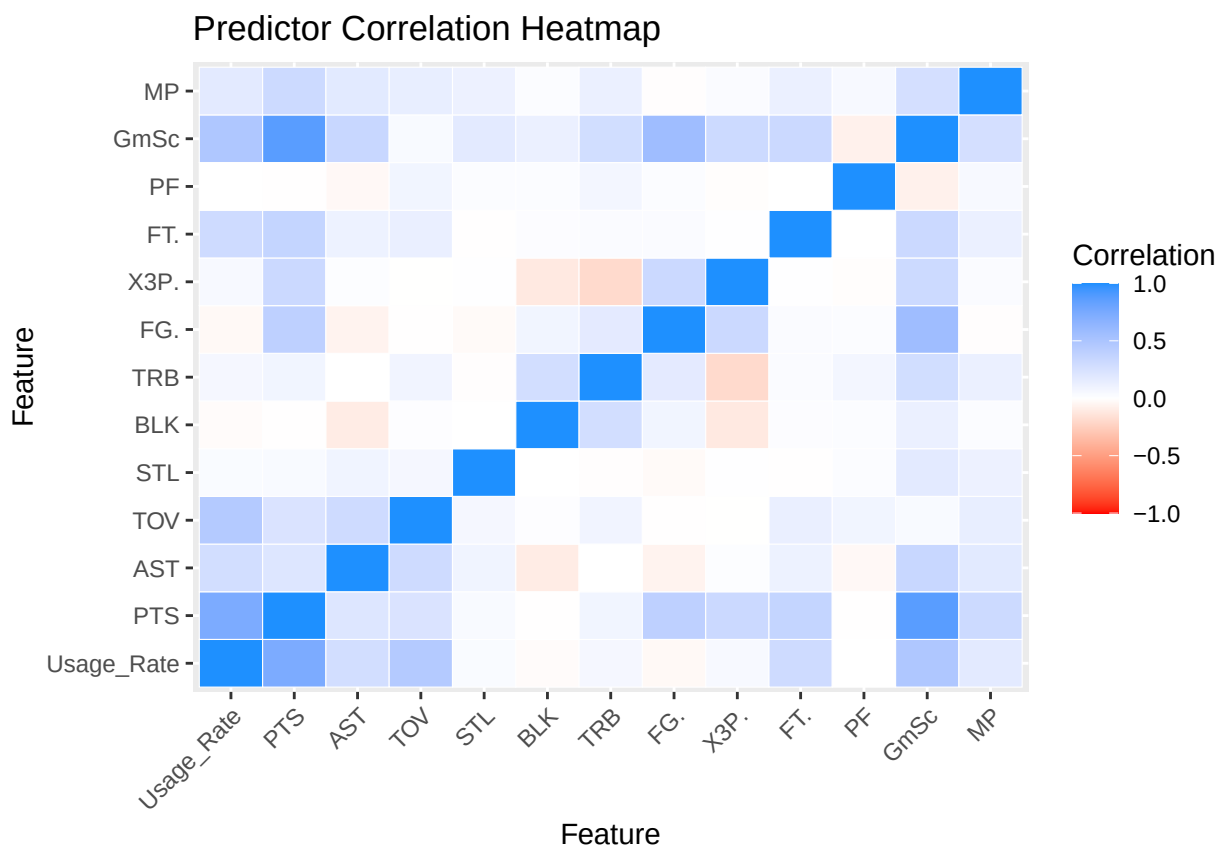

Figure 4: Correlation Heatmap for All NBA Win Predictors

The results of the heatmap have a major impact on how we interpret the features. Specifically, the heatmap shows how the Game Score, Points, and Usage Rate all have a strong correlation. This confirmed the risk that I initially mentioned, that there is collinearity here. These features are typically tightly related to each other, and the model will therefore struggle with deciding which gets more credit. For example, in my SHAP waterfall plot for Jayson Tatum, the Game Score had a massive positive SHAP value while Points got a negative SHAP value. In this context, this doesn't mean points are necessarily harmful but the model already gave most of the credit of the win probability to the Game Score. Additionally, there is definitely a

risk of extrapolation, specifically for my CP Profiles. With the Usage Rate and Points being so correlated, the Usage Rate will increase in percentage while the Points will decrease, creating a player who uses most of the team's possession while scoring less points which is something that will become increasingly less likely to happen (low real world examples). Because of this correlation, the SHAP might assign a non-zero value to an irrelevant feature just because it is correlated with a predictor that is genuine in the test set.

Focusing specifically on the SHAP method, the main hyperparameter is the "link" function which controlled where the SHAP values are calculated on a log-odds scale (ex. for the XGBoost model) or the probability scale. By default, for the SHAP value calculation, the log-odds scale is used and is necessary because it ensures that the feature contributions are additive.

```r
set.seed(456)

# log-odds scale
log_odds_val <- first_game$S[,"Usage_Rate"]

# transformation
prob_impact <- 1 / (1 + exp(-log_odds_val))

log_odds_val
```

```
##  Usage_Rate
## -0.07268795
```

```r
prob_impact
```

```
## Usage_Rate
##   0.481836
```

In the code above, I specifically tested the sensitivity of the Usage Rate variable for Jayson Tatum comparing the log-odds scale versus the probability scale. The contribution on the log-odds scale is about -0.073 which is about 0.4818 on the probability scale after the logistic transformation. This showcases how the link function acts a non-linear notifier where small changes in the log-odds don't translate linearly in the probability. The log-odds provide more of a pure view of a feature's impact (through additivity) on the model whereas the probability scale is better for showing the true risk of high-usage to a coach.

Finally, between the two methods (CP Profile and SHAP), there were multiple similarities but also major differences I found in the context of sports analytics. For the CP Profile and the SHAP Global Importance method, the Usage Rate and Game Score were found as the most vital predictors, showcasing that both methods exemplify that there is a negative impact on too high of a usage. On the other hand, the main difference was that the CP Profile was better for advice that is more for what should be put into action (such as we can tell a coach that they should target a specific usage percentage for a better team win probability for their specific team) while the SHAP profile method was better for a diagnosis, such as noticing that Tatum's usage was high but his impact on team efficiency was still high enough for him to have a positive impact on the game as a whole.